SME Cash Shock Simulator: Build Spec

Oct 9, 2026 · @Jun Ze

Purpose and scope

Build an interactive simulator that tells a small-business owner whether they survive a cash shock, which sequence of actions gives the best outcome, and the largest shock they can survive. The simulator is general: it reads any business described in the input format (any number of customers, payables, credit lines and decision points) and runs it. The hand-drawn business in this spec is one worked example used to check the engine, not the only scenario the tool handles. Every number in the input, including the shock, is a variable the user can change; nothing is hard-coded to the example.

The prototype must demonstrate four things:

1. **Timeline:** a day-by-day cash timeline built from receivables, payables, new sales and financing.
2. **Branching:** at each decision point, every available action is simulated, and the branches are shown as a tree.
3. **Best path:** the branch with the best score is highlighted; all other branches are drawn semi-transparent (about 30% opacity).
4. **Survival threshold:** for a user-chosen shock (a variable, defaulting to S$30,000 due within 5 days of day 35), the probability of survival, the largest survivable shock at 95% confidence, and the extra cash buffer needed to survive the chosen shock.

Uncertain things (when customers pay, whether they take a discount, whether they leave after a hard collection action) are drawn from probability distributions. Every result comes from many simulated runs (default 1,000 per branch), not one guess.

The LLM agent that sets objectives and explains results to the owner is **out of scope**. The simulator must expose a clean JSON input and output so the agent can be added later.

Rules for the implementer. Build exactly what this spec says. If a behaviour is not specified, or two statements seem to conflict, stop and ask rather than choosing. Do not add features, defaults, fallbacks or input fields that are not written here. Where the spec gives a formula, pseudocode or an expected test value, it is the definition; prose around it is explanation only.

Assumptions to confirm

The hand-drawn sheet leaves five points open. The build uses the defaults below; each is a config value, so changing one needs no code change.

Item on the sheet	Default used	Alternative reading
Credit terms "5% / 30" and "2% / 45"	5% (or 2%) off if paid within the window; full amount due 60 days after invoice	Window and due date are the same day
Days to payment	Expected days from day 0 until payment; firm B's −4 means 4 days overdue already, which is why it has manual chasing	B pays 4 days early
Interest "2% for max S$5,000"	Credit line with a S$5,000 limit at 2% per month on the drawn balance	2% per year
Revenue S$700 vs receivables S$2,480	Receivables are older invoices; S$700 is new monthly sales, repeating each month on each customer's terms	Revenue is already inside receivables
Machine price cut "−20% price, +12% quantity"	Price elasticity 0.6 (12 / 20); baseline is the revenue table (6 machines, 20 parts)	Different baseline quantity

Two corrections from the hand calculation are built in:

- Firm A paying inside its discount window pays S$1,425 (5% off S$1,500), not S$1,470.
- A 20% price cut with a 12% volume gain lowers machine revenue: 6 × S$100 = S$600 becomes 6.72 × S$80 = S$537.60. The simulator must report changes against the baseline, never the new total as a gain.

Input data model

Nothing in the engine is specific to this example. Every quantity below is a field in `scenario.json`, and the engine iterates over whatever lists it is given: N customers, M payables, any number of credit lines, any set of decision points and actions. Loading a different `scenario.json` must change the whole run with no code change. The sample below is one such file, used only to check the engine against a hand calculation.

The only things that live in code, not config, are the hard safety limits (cash cannot go below zero once every credit line is exhausted; a credit line cannot be drawn past its limit) and the event ordering within a day. Everything else, including which decision points exist and what each action does, is data.

All inputs live in one JSON file, `scenario.json`. Money is in S$; time is in whole days from day 0. The sample below is the hand-drawn business.

```
{
  "meta": { "currency": "SGD", "horizonDays": 90, "runsPerPath": 1000,
"seed": 42 },
  "business": { "cash0": 20000, "grossMargin": 0.30, "creditRepayAbove": 3000
},
  "priceElasticity": 0.6,
  "customers": [
    { "id": "A", "name": "Firm A", "keyClient": true, "meanDaysToPay": 180,
      "dueDays": 60, "earlyPayDiscount": { "rate": 0.05, "windowDays": 30 },
      "offerAcceptance": { "alpha": 1, "beta": 9 },
      "orderCycleDays": 30, "firstOrderDay": 30,
      "orders": [ { "item": "machine", "unitPrice": 100, "qtyPerCycle": 5 },
                  { "item": "parts", "unitPrice": 5, "qtyPerCycle": 20 } ] },
    { "id": "B", "name": "Firm B", "keyClient": false, "meanDaysToPay": 10,
      "dueDays": 60, "earlyPayDiscount": { "rate": 0.02, "windowDays": 45 },
      "offerAcceptance": { "alpha": 6, "beta": 4 },
      "orderCycleDays": 30, "firstOrderDay": 30,
      "orders": [ { "item": "machine", "unitPrice": 100, "qtyPerCycle": 1 } ]
},
    { "id": "C", "name": "Firm C", "keyClient": false, "meanDaysToPay": 30,
      "dueDays": 60, "earlyPayDiscount": { "rate": 0.02, "windowDays": 45 },
      "offerAcceptance": { "alpha": 6, "beta": 4 },
      "orderCycleDays": 30, "firstOrderDay": 30, "orders": [] }
  ],
  "receivables": [
    { "id": "AR-A1", "customerId": "A", "amount": 1500, "invoiceDay": 0 },
    { "id": "AR-B1", "customerId": "B", "amount": 780,  "invoiceDay": -64 },
    { "id": "AR-C1", "customerId": "C", "amount": 200,  "invoiceDay": -15 }
  ],
  "payables": [
    { "id": "AP-UTIL", "label": "Utilities", "amount": 500, "dueDay": 23,
      "affectsProfit": true, "earlyPayDiscount": null },
    { "id": "AP-INV", "label": "Inventory", "amount": 10000, "dueDay": 60,
      "affectsProfit": false, "earlyPayDiscount": { "rate": 0.10, "byDay": 45
} }
  ],
  "recurringCosts": [],
  "creditLines": [ { "id": "LINE1", "limit": 5000, "ratePerMonth": 0.02 } ],
```

```
  "shocks": [ { "id": "SHOCK1", "label": "Emergency cash need", "amount":
30000,
                "announceDay": 35, "dueDay": 40, "affectsProfit": true } ],
  "actions": [
    { "id": "none",     "type": "collection", "label": "Do nothing",
      "payProbMultiplier": 1.0, "cashCost": 0, "lateFeeRatePerMonth": 0,
      "churnProb": 0, "churnProbKeyClient": 0 },
    { "id": "resend",   "type": "collection", "label": "Resend invoice",
      "payProbMultiplier": 1.5, "cashCost": 0, "lateFeeRatePerMonth": 0,
      "churnProb": 0, "churnProbKeyClient": 0 },
    { "id": "late_fee", "type": "collection", "label": "Charge late fee",
      "payProbMultiplier": 3.0, "cashCost": 0, "lateFeeRatePerMonth": 0.01,
      "churnProb": 0.05, "churnProbKeyClient": 0.10 },
    { "id": "letter",   "type": "collection", "label": "Letter of demand",
      "payProbMultiplier": 10.0, "cashCost": 200, "lateFeeRatePerMonth": 0,
      "churnProb": 0.20, "churnProbKeyClient": 0.40 },
    { "id": "offer",    "type": "discount_offer", "label": "10% off if paid
in 2 days",
      "offerRate": 0.10, "payWithinDays": 2 },
    { "id": "price_cut", "type": "price_change", "label": "Cut prices 20%",
      "priceMultiplier": 0.8 },
    { "id": "pay_early", "type": "payable_timing", "label": "Pay early for
discount",
      "payEarly": true },
    { "id": "pay_due",  "type": "payable_timing", "label": "Pay on due date",
      "payEarly": false }
  ],
  "decisionPoints": [
    { "id": "D1", "day": 10, "order": 1, "label": "Chase Firm A", "kind":
"single",
      "options": [ { "actionId": "none", "targetId": "A", "isDefault": true
},
                   { "actionId": "resend", "targetId": "A" },
                   { "actionId": "late_fee", "targetId": "A" },
                   { "actionId": "letter", "targetId": "A" } ] },
    { "id": "D2", "day": 10, "order": 2, "label": "Chase Firm B", "kind":
"single",
      "options": [ { "actionId": "none", "targetId": "B", "isDefault": true
},
                   { "actionId": "late_fee", "targetId": "B" },
                   { "actionId": "letter", "targetId": "B" } ] },
    { "id": "D3", "day": 35, "order": 1, "label": "Respond to shock", "kind":
"subset",
      "options": [ { "actionId": "offer", "targetId": "ALL_OPEN_RECEIVABLES"
},
                   { "actionId": "letter", "targetId": "B" },
```

```
                  { "actionId": "price_cut", "targetId": "ALL_CUSTOMERS" } ]
},
    { "id": "D4", "day": 45, "order": 1, "label": "Inventory bill", "kind":
"single",
      "options": [ { "actionId": "pay_due", "targetId": "AP-INV",
"isDefault": true },
                   { "actionId": "pay_early", "targetId": "AP-INV" } ] }
  ],
  "objective": { "survivalTarget": 0.95, "churnHorizonMonthsAfterEnd": 9,
                 "profitTieToleranceSGD": 1.0 },
  "threshold": { "shockIdToScale": "SHOCK1", "maxShock": 50000,
"maxExtraBuffer": 40000,
                 "stepSGD": 500, "runsPerPath": 300, "curveStepSGD": 2500 }
}
```

Field reference

Every field is required unless marked optional. Validate the file on load and refuse to run with a message naming the bad field; never fill a missing field with a guess.

Field	Type and range	Meaning
`meta.horizonDays`	integer, 1–365	Days simulated. Day 0 is the opening state; the loop runs days 1 to horizonDays inclusive.
`meta.runsPerPath`	integer, 100–10,000	Monte Carlo runs per path.
`meta.seed`	32-bit unsigned integer	Base seed for every random draw.
`business.cash0`	number, S$	Cash at the end of day 0.
`business.grossMargin`	0–1	Share of invoiced revenue kept after cost of goods.
`business.creditRepayAbove`	number ≥ 0, S$	Credit lines are repaid only with cash above this level.

Field	Type and range	Meaning
`priceElasticity`	number ⩾ 0	Volume multiplier = max(0, 1 + elasticity × (1 − priceMultiplier)). A 20% cut with elasticity 0.6 gives × 1.12.
`customers[].meanDaysToPay`	number ⩾ 1	Mean days until an open invoice is paid, with no action taken. Daily pay probability h = 1 / meanDaysToPay. Same value applies to existing and new invoices.
`customers[].dueDays`	integer ⩾ 0	Invoice falls due on invoiceDay + dueDays.
`customers[].earlyPayDiscount`	object or null	Customer pays amount × (1 − rate) if paid on or before invoiceDay + windowDays.
`customers[].offerAcceptance`	alpha > 0, beta > 0	Beta prior; acceptance probability of a discount offer = alpha / (alpha + beta).
`customers[].orderCycleDays`, `firstOrderDay`	integers ⩾ 1	New invoices are raised on firstOrderDay, firstOrderDay + cycle, … up to horizonDays.
`customers[].orders[]`	unitPrice ⩾ 0, qtyPerCycle ⩾ 0	One line per item invoiced each cycle.
`receivables[]`	amount > 0, invoiceDay ⩽ 0	Invoices open at day 0. A negative invoiceDay means issued before day 0.
`payables[]`	amount > 0, dueDay 1–horizonDays	Bills open at day 0. `affectsProfit: true` = operating expense; `false` = stock purchase already counted through gross margin.

Field	Type and range	Meaning
`payables[].earlyPayDiscount`	object or null	Pay amount × (1 − rate) on byDay if the pay_early action is chosen.
`recurringCosts[]`	id, label, amount, firstDay, repeatEveryDays, affectsProfit	Costs paid on firstDay and every repeatEveryDays after, within the horizon.
`creditLines[]`	limit ⩾ 0, ratePerMonth ⩾ 0	Drawn in list order, repaid in reverse list order.
`shocks[]`	amount ⩾ 0, announceDay ⩽ dueDay	Cash outflow paid in full on dueDay. announceDay is shown on the timeline only.
`actions[]`	see action types section	Reusable action definitions.
`decisionPoints[]`	see action types section	When choices are made and which options exist.
`objective`, `threshold`	see objective and optimiser sections	Ranking and threshold search settings.

The example values for firm B (meanDaysToPay 10, invoiceDay −64 so it is 4 days past its day-60 due date) and the invoice days for firms A and C are owner estimates for this example only.

Timeline, events and decision points

The simulator steps one day at a time from day 0 to `horizonDays`, but decisions happen only at a few named decision points. Deciding every day would create about 3^78 paths; deciding at events keeps the tree small enough to simulate every branch.

Events happen on their own and are not choices:

- A payable falls due (utilities day 23, inventory day 60).
- A customer pays (random day, see the probability models).
- New monthly sales are invoiced (every 30 days, from the order table).
- The shock is announced (day 35) and falls due (day 40).

- Credit-line interest is charged (daily accrual, charged monthly).

Decision points for the sample business, in order:

Node	Day	Decision	Options
D1	10	Collection action on the most overdue key client (firm A)	none, resend invoice, late fee, letter of demand
D2	10	Collection action on firm B (already overdue)	none, late fee, letter of demand
D3	35	Response to the shock	any subset of: discount offer to all debtors, pressure firm B, price cut
D4	45	Inventory payable	pay early for 10% off, pay on day 60

D3 is a subset choice, so it has 2^3 = 8 options. The full tree is 4 × 3 × 8 × 2 = 192 paths. At 1,000 runs each, that is 192,000 runs of 90 days, which takes seconds in the browser.

The credit line is not a decision node. It is drawn automatically whenever cash would go below zero, up to its limit, and repaid automatically when cash exceeds a configurable level (default S$3,000). This rule replaces 90 separate borrowing decisions with one setting.

Decision points must come from config, not hard-coded logic, so a different business can define its own nodes.

Action types and decision points

There are exactly four action types. Each option at a decision point applies one action to one target. All effects start on the decision day and last to the end of the horizon unless stated.

Type	Valid targetId	Effect when applied on day t
`collection`	a customer id	1. Multiply that customer's collection multiplier by `payProbMultiplier`. 2. If `lateFeeRatePerMonth` > 0, set the customer's late-fee rate to the larger of its current rate and this one, with fee start day t (kept if already set). 3. Pay `cashCost` now: cash −= cashCost; collectionCosts += cashCost. 4. Churn draw: churn probability = `churnProbKeyClient` if the customer is a key client, else `churnProb`. If u < p, the customer is churned for the rest of the run.
`discount_offer`	`ALL_OPEN_RECEIVABLES`	For each open invoice, in invoice id order: accept if u < alpha / (alpha + beta) of its customer. An accepted invoice is paid on day t + `payWithinDays` at amount × (1 − `offerRate`), and gets no hazard draws before then. If the invoice's own early-pay discount is larger and still valid on that day, use the larger one; never both.
`price_change`	`ALL_CUSTOMERS` or a customer id	Set the price multiplier to `priceMultiplier` for every invoice raised on day t or later for the target. Volume multiplier = max(0, 1 + priceElasticity × (1 − priceMultiplier)). Quantities stay fractional (expected units).
`payable_timing`	a payable id	`payEarly: true` and t ⩽ byDay: pay amount × (1 − rate) on byDay. `payEarly: false`, or t > byDay: pay full amount on dueDay. A payable with no decision point is paid in full on dueDay.

Action rules:

- Applying any action to a customer, invoice or payable that is already settled or churned does nothing: no cost, no draw.

- Collection multipliers stack by multiplication. Effective daily pay probability = min(0.95, h × multiplier).
- A churned customer still pays its open invoices as normal. It gets no new invoices.
- Late fee on an invoice paid on day t_pay = amount × rate × max(0, t_pay − max(dueDay, feeStartDay)) / 30. The fee is added to the cash received and recorded as lateFeeIncome. On the example, firm B (due day −4) charged from day 10 and paying on day 20 owes S$780 × 0.01 × 10 / 30 = S$2.60.

Decision point rules:

- `kind: "single"` = choose exactly one option. `kind: "subset"` = choose any subset, including none; options in a subset are applied in their listed order.
- Decision points on the same day are applied in `order`, ascending.
- The default path takes the `isDefault` option at every single-choice point and the empty set at every subset point. It is the "do nothing" baseline for profit sacrificed. Exactly one option per single-choice point must have `isDefault: true`.
- Subset points may have at most 6 options. If the total number of paths exceeds 5,000, refuse to run and show the count.

Probability models

Four uncertain things are modelled. Each starts from a prior (a standard default or the owner's estimate) and can later be updated from the business's own history. For the prototype, priors only.

1. When each customer pays: a daily hazard

Each unpaid invoice has a probability h of being paid on any given day. The expected wait is 1 / h days, so h is set from the sheet's days-to-pay:

```
h_{\text{base}} = \frac{1}{\max(\text{expected days to pay}, 1)}
```

Customer (example file)	meanDaysToPay	Daily h
Firm A	180	0.00556
Firm B	10	0.100
Firm C	30	0.0333

Each day, each open invoice that is not under an accepted offer and was not raised today gets one draw u. It is paid today if u < min(0.95, h × the customer's collection multiplier). The model is memoryless: the chance of paying tomorrow does not depend on how long the invoice has been open, so the same h serves existing and new invoices. An invoice can therefore be paid before its due date; that is intended.

2. Whether a customer takes a discount

If a customer pays inside a discount window, it pays the discounted amount. For the shock-time offer (10% off if paid in 2 days), each customer accepts with probability p:

Customer	p (accept 10% offer)	Prior
Firm A	0.10	Beta(1, 9)
Firm B	0.60	Beta(6, 4)
Firm C	0.60	Beta(6, 4)

Store each p as a Beta(a, b) so it can be updated: after k acceptances in n past offers, the estimate is (a + k) / (a + b + n).

3. Goodwill as churn

Each collection action carries a churn probability (`churnProb`, or `churnProbKeyClient` for key clients). Draw once, on the day the action is applied. A churned customer gets no new invoices for the rest of the run, which lowers revenue and profit inside the horizon. Margin lost after the horizon is charged separately as churn cost (objective section), so nothing is counted twice.

4. Price response

New invoice volume = base quantity × max(0, 1 + elasticity × (1 − priceMultiplier)). This linear form matches the owner's estimate exactly: a 20% cut with elasticity 0.6 gives 12% more units. With elasticity below 1, a price cut always lowers revenue; the simulator shows that rather than hiding it. Firm A after a 20% cut: 5 × 1.12 × S$80 + 20 × 1.12 × S$4 = S$537.60 per cycle, down from S$600.

Common random numbers

Every path must see the same random numbers for run i, so differences between paths come only from actions. Do not draw from a shared stream in loop order; key every draw by what it is for:

```
u = uniform(seed, run, streamKey, day)
streamKey examples:
  "pay:" + invoiceId                       // daily payment draw
  "churn:" + customerId + ":" + decisionId  // churn draw for one action
  "accept:" + invoiceId + ":" + decisionId  // offer acceptance draw
```

Implement `uniform` as: h = fnv1a32(streamKey); s = (seed ^ imul(run + 1, 0x9E3779B1) ^ h ^ imul(day + 1, 0x85EBCA77)) >>> 0; return the first output of mulberry32(s) divided by 2^32. New invoice ids are deterministic: `customerId + "-INV-" + day`. With this keying, the same invoice on the same day gets the same draw in every path.

Cash accounting rules

Day 0 is the opening state from the input file; no events run on day 0. The engine then runs days t = 1 to horizonDays in this exact order. Any rule not listed here does not exist.

```
for t in 1..horizonDays:
  if failed: copy previous snapshot with day = t; continue

  1. DECISIONS: for each decision point with day == t, ascending order:
       apply the path's chosen option(s) per the action types section
       (cashCost is paid immediately; cash may go negative here, step 5 fixes
it)

  2. NEW INVOICES: for each customer, not churned, where
       t >= firstOrderDay and (t - firstOrderDay) % orderCycleDays == 0:
         amount = sum(unitPrice * priceMult * qtyPerCycle * max(0, 1 +
elasticity * (1 - priceMult)))
         if amount > 0: add invoice {id: cust-INV-t, amount, invoiceDay: t}
         revenue += amount;  cogs += amount * (1 - grossMargin)

  3. COLLECTIONS, in invoice id order (skip invoices raised today):
       if accepted offer and t == offerPayDay: pay offer amount
       elif not accepted and u("pay:"+id, t) < min(0.95, h * mult): pay
       paid principal = amount * (1 - best valid discount rate)   // window
or offer, never both
       fee = late fee per the action types section
       cash += principal + fee;  discountsGiven += amount - principal
       lateFeeIncome += fee;  remove the invoice from receivables

  4. OUTFLOWS, in this order:
       payables scheduled today (dueDay, or byDay if paying early):
         cash -= paid; supplierDiscounts += amount - paid
         if affectsProfit: operatingExpense += paid
```

```
          remove the payable from payables
       recurring costs due today: cash -= amount; if affectsProfit:
operatingExpense += amount
       shocks with dueDay == t: cash -= amount; if affectsProfit:
shockExpense += amount

  5. CREDIT: if cash < 0: draw from lines in list order up to each limit
       if cash still < 0: failed = true; failDay = t
     elif cash > creditRepayAbove: repay lines in reverse order
       with (cash - creditRepayAbove), never more than drawn

  6. INTEREST: accrued += sum(drawn_i * ratePerMonth_i / 30)
     if t % 30 == 0 or t == horizonDays:
       cash -= accrued; interestPaid += accrued; accrued = 0
       if cash < 0: repeat step 5 once

  7. SNAPSHOT: record the state fields below for day t
```

State recorded per day: `day, cash, creditDrawn, accruedInterest, receivablesOpen, payablesOpen, revenue, cogs, discountsGiven, lateFeeIncome, supplierDiscounts, collectionCosts, operatingExpense, shockExpense, interestPaid, profit, failed, events[]`. All money values are rounded to cents only when written to output, never during the run.

Failure: a run fails on the first day cash is below zero after every credit line is fully drawn. Its later days repeat the failure-day snapshot. Failed runs stay in every average and percentile with their frozen values; the dashboard also shows the same metrics for surviving runs only.

Accounting identity, asserted every day in tests, to within S$0.01:

```
net = cash - creditDrawn - accruedInterest - payablesOpen + receivablesOpen
net(t) - net(t-1) = invoicesRaised - discountsGiven + lateFeeIncome
                  + supplierDiscounts - collectionCosts - interestAccrued
                  - recurringCostsPaid - shocksPaid
```

(all terms on the right are today's amounts; paying a payable or collecting an invoice moves money between terms of net and leaves it unchanged except for discounts and fees.)

Deterministic mode replaces each draw with a fixed rule, so results can be checked by hand: an invoice keeps a survival value S starting at 1, multiplied each eligible day by (1 − effective p); it is paid on the first day S ⩽ 0.5. Churn happens if p ⩾ 0.5. An offer is accepted if alpha / (alpha + beta) ⩾ 0.5. Deterministic mode runs once per path, not runsPerPath times.

Time value of money is left out of the prototype. Over 90 days at 2% a month it changes results far less than late payments and discounts do. Add a discount-rate setting later if needed.

Objective, constraints and goodwill cost

Paths are ranked by a fixed order of keys, not a weighted score. Survival is the hard requirement and profit is the value; adding cash and profit into one number would count each discount twice, because a discount lowers both.

Profit definitions (per run, over the horizon)

```
grossProfit      = revenue - cogs
interestExpense  = interestPaid + accruedInterest at end
operatingProfit  = grossProfit - discountsGiven + lateFeeIncome +
supplierDiscounts
                 - collectionCosts - operatingExpense - interestExpense
profit           = operatingProfit - shockExpense
churnCost        = sum over churned customers c of monthlyMargin_c *
churnHorizonMonthsAfterEnd
monthlyMargin_c  = sum(unitPrice * qtyPerCycle) * (30 / orderCycleDays) *
grossMargin   // at base price
economicProfit   = profit - churnCost
endNetCash       = cash - creditDrawn - accruedInterest   // on the last day
```

Rules that follow from these definitions:

- Revenue is recognised when an invoice is raised. Invoices open at day 0 were recognised before day 0, so they add no revenue, but discounts and late fees on them do count.
- A payable with `affectsProfit: false` (stock) is cash only, because its cost is already inside COGS. Its early-pay discount still counts as supplierDiscounts.
- Churn cost covers only the months after the horizon. Lost orders inside the horizon already show up as lower revenue. Example: firm A's monthly margin is S$600 × 0.30 = S$180, so churn costs S$180 × 9 = S$1,620.

Ranking

For each path, compute over all runs: survivalProb (share of runs not failed), mean economicProfit, mean endNetCash.

1. A path is **eligible** if survivalProb ≥ `objective.survivalTarget`.

2. If at least one path is eligible, the best path is the eligible path with the highest mean economicProfit. Ties (within `profitTieToleranceSGD`) are broken by higher survivalProb, then higher mean endNetCash, then path id in alphabetical order.
3. If no path is eligible, the best path is the one with the highest survivalProb. Ties (equal counts of surviving runs) are broken by mean economicProfit, then mean endNetCash, then path id.

Profit sacrificed and the profit bridge

- **Profit sacrificed** = mean economicProfit of the default path − mean economicProfit of the best path. Positive means survival cost money; negative means the best plan also earns more.
- **Bridge items** (each is mean of best path − mean of default path): grossProfit, −discountsGiven, lateFeeIncome, supplierDiscounts, −collectionCosts, −operatingExpense, −interestExpense, −shockExpense, −churnCost. They add up exactly to the difference in mean economicProfit; assert this in tests.

Frontier

Also output the **frontier**: the paths not beaten by any other path on both survivalProb and mean economicProfit at once. It shows the owner the real trade-off (for example, 97% survival with S$800 profit versus 91% with S$1,400) instead of only the single winner.

Hard limits are always enforced in code, whatever the config says: no credit drawn beyond a line's limit, and a run fails when cash is below zero after every line is fully drawn.

Optimiser and threshold search

The prototype simulates every path (full enumeration) and ranks them with the objective section's rules. No learning, no sampling of paths.

Main run

```
paths = cartesian product of each decision point's options
        (subset points contribute every subset, empty set first, then by
size, then listed order)
pathId = join("|", for each decision point in (day, order): id + ":" + chosen
option ids joined by "+" or "none")
for each path:
  for run in 0..runsPerPath-1: trace = simulate(scenario, path, run)
  summarise -> survivalProb, means, percentiles, fan, medianRun
rank -> bestPathId, frontier
```

- **Percentiles** use nearest rank: sort the n values ascending; the p-th percentile is the value at index ceil(p / 100 × n) − 1.
- **Fan** = p10, p50 and p90 of each recorded field on each day, across runs.
- **Median run** = the run whose endNetCash is closest to the path's p50 endNetCash; ties go to the lowest run index. Its full daily trace and events are output.
- **Per-edge summary** for the tree: for each decision point and option, the best-ranked path that takes that option (its pathId, survivalProb and mean economicProfit).

Largest survivable shock

Scale only the shock named in `threshold.shockIdToScale`. Use `threshold.runsPerPath` runs per path and the same seed. "Best survival at amount S" means: re-run every path with the shock set to S, rank, and take the best path's survivalProb.

1. If best survival at `maxShock` ⩾ target: report S* = maxShock with flag `atLeast: true`.
2. If best survival at 0 < target: report S* = null ("target not met even with no shock").
3. Otherwise bisect on the grid of multiples of `stepSGD`: lo = 0 (passes), hi = maxShock (fails). Test the grid point nearest the midpoint, move lo or hi, stop when hi − lo = stepSGD. Report S* = lo.
4. Output best survival at S* and at S* + stepSGD, so a non-monotone result is visible rather than hidden.

Extra buffer for the chosen shock

Keep the shock at its input amount and add Δ to `cash0`. Same procedure on Δ from 0 to `maxExtraBuffer`: report 0 if the target is met with no extra cash; report null ("more than maxExtraBuffer") if it is not met at the maximum; otherwise the smallest passing grid value.

Survival curve

Hold the main run's best path fixed. Compute its survivalProb at shock amounts 0, curveStepSGD, 2 × curveStepSGD, …, maxShock, with `meta.runsPerPath` runs each.

Time budget and progress

The example (192 paths) must finish main run, both searches and the curve within 15 seconds in Chrome on a mid-range laptop. The worker posts progress after each stage: `main`, `shock_search`, `buffer_search`, `curve`, `done`, each with percent complete.

Sanity bound for the example

By day 40 the most cash the example can hold is S$20,000 − S$500 utilities + S$2,480 of open invoices + S$700 of day-30 invoices + S$5,000 credit = S$27,680, before any discount or lawyer cost. So the S$30,000 shock must fail in every run of every path, and the result must say so, with the extra buffer reported as a positive number.

Output data contract

The engine returns one `Result` object. The UI reads only this object and never recomputes a metric. Money is a number in S$ rounded to 2 decimals; probabilities are numbers from 0 to 1 rounded to 4 decimals. Every daily array has length horizonDays + 1 (index = day).

```
type Pct = { p10: number; p50: number; p90: number };
type DailyPct = { p10: number[]; p50: number[]; p90: number[] };

type Metrics = {                      // means over runs unless named
p10/p50/p90
  survivalProb: number;
  economicProfit: Pct & { mean: number };
  profit: Pct & { mean: number };
  endNetCash: Pct & { mean: number };
  revenue: number; grossProfit: number; discountsGiven: number;
  lateFeeIncome: number; supplierDiscounts: number; collectionCosts: number;
  operatingExpense: number; interestExpense: number; shockExpense: number;
  churnCost: number; peakCreditDrawn: number;
  endReceivables: number; endPayables: number;
  failDayHistogram: Record<string, number>;   // day -> number of runs
failing that day
};

type Event = { type: "invoice" | "collection" | "payable" | "recurring" |
"shock"
                  | "credit_draw" | "credit_repay" | "interest" | "decision"
| "churn" | "failure";
               label: string; amount: number; refId: string };

type DaySnapshot = { day: number; cash: number; creditDrawn: number;
receivablesOpen: number;
                     payablesOpen: number; revenue: number; profit: number;
failed: boolean;
                     events: Event[] };

type PathResult = {
```

```
  id: string;
  choices: Record<string, string[]>;   // decisionPointId -> chosen option
ids ([] = none)
  isDefault: boolean;
  metrics: Metrics;                    // all runs
  metricsSurvivorsOnly: Metrics | null;// null if no run survived
  daily: { cash: DailyPct; creditDrawn: DailyPct; receivablesOpen: DailyPct;
           payablesOpen: DailyPct; revenue: DailyPct; profit: DailyPct };
  medianRun: DaySnapshot[];
};

type Result = {
  scenarioName: string;
  bestPathId: string;
  defaultPathId: string;
  bestMeetsTarget: boolean;
  paths: PathResult[];                 // every path, sorted by rank, best
first
  frontierPathIds: string[];
  profitSacrificed: number;
  profitBridge: { item: string; amount: number }[];   // order as in the
objective section
  tree: {
    nodes: { id: string; day: number; order: number; label: string; kind:
"single" | "subset" }[];
    edges: { nodeId: string; optionId: string; label: string; bestPathId:
string;
             survivalProb: number; economicProfit: number; onBestPath:
boolean }[];
  };
  threshold: {
    largestSurvivableShock: number | null; atLeast: boolean;
    survivalAtThreshold: number | null; survivalAtNextStep: number | null;
    extraBufferForChosenShock: number | null;
    curve: { shock: number; survival: number }[];
  };
  runInfo: { runsPerPath: number; thresholdRunsPerPath: number; seed: number;
             pathCount: number; elapsedMs: number; deterministic: boolean };
};
```

For a `subset` decision point, the tree has one edge per subset (8 edges for 3 options), labelled with the option labels joined by " + ", or "No action" for the empty set.

Dashboard UI

One page, five panels stacked top to bottom in this order. At widths of 1,024px and above, Panel 5 (controls) sits in a 320px left sidebar and Panels 1–4 fill the right. Below 1,024px everything is one column with controls last. Charts use Recharts; the decision tree is hand-drawn SVG. Every element below names the `Result` field it reads; the UI computes nothing itself.

Fixed styling

Series or state	Colour
Cash	#2563EB
Profit	#16A34A
Revenue	#7C3AED
Receivables	#0891B2
Payables	#EA580C
Credit drawn	#F59E0B
Shock and failure	#DC2626
Pinned alternative path	#6B7280, dashed

Money format: prefix `S$`, thousands separators, no decimals in tiles and axes, 2 decimals in event cards and tooltips. Negative money is shown with a leading minus sign in red text (for example `-S$1,234`). Probabilities: percent with 1 decimal.

Panel 1: Verdict

- Badge from the best path's `metrics.survivalProb` p: "Survives" (green) if p ≥ survivalTarget; "At risk" (amber) if 0.5 ≤ p < target; "Fails" (red) if p < 0.5. Badge text includes p, for example "Fails — 0.0% of runs survive".
- Four tiles: largest survivable shock (`threshold.largestSurvivableShock`; prefix "≥" if `atLeast`; "None" if null), extra buffer needed (`extraBufferForChosenShock`; "More than S$40,000" if null), profit sacrificed (`profitSacrificed`), expected churn cost (`metrics.churnCost`).
- Plan sentence built from the best path's choices in day order, using option labels: "Day 10: Charge late fee (Firm A). Day 10: Letter of demand (Firm B). Day 35: No action. Day 45: Pay on due date (Inventory)." No free text, no LLM.

- If `bestMeetsTarget` is false, show one line under the badge: "No plan meets the 95% target; showing the plan with the highest survival."
- Survival curve: small line chart of `threshold.curve`, x = shock amount, y = survival (0–100%), horizontal dashed line at the target, vertical dashed line at the chosen shock amount.

Panel 2: Cash timeline

- Line chart, x = day 0 to horizonDays, y = cash. Solid cash line = `medianRun[].cash`; shaded band = `daily.cash` p10 to p90 at 20% opacity.
- Reference lines: y = 0 (solid grey) and y = minus the total credit limit (dashed grey, labelled "Credit limit"). Shock window: red shaded rectangle from announceDay to dueDay.
- Event markers on the cash line from `medianRun[].events`: collection = upward triangle, payable or recurring = downward triangle, invoice = small circle, credit draw = amber square, failure = red cross. Hover shows the event label and amount.
- Scrubber under the chart: slider for day plus Play/Pause (advances 1 day every 150ms). The selected day draws a vertical line and fills an event card: day, cash, credit drawn, receivables, payables, and that day's events.

Panel 3: Financial dashboard

- Nine tiles, each showing the mean and "p10–p90" underneath where a Pct exists: Revenue, Gross profit, Profit (`profit`), Economic profit (`economicProfit`), End receivables, End payables, Peak credit drawn, Discounts given, Interest expense.
- Chart A, "Revenue and profit to date": lines from `daily.revenue.p50` and `daily.profit.p50`; band on profit p10–p90.
- Chart B, "Money owed to you vs bills": lines from `daily.receivablesOpen.p50` and `daily.payablesOpen.p50`.
- Chart C, "Cash and credit": lines from `daily.cash.p50` and `daily.creditDrawn.p50`, no event markers.
- Chart D, "What survival cost": waterfall from `profitBridge`. First bar = default path's economic profit, then one floating bar per bridge item (green if positive, red if negative), last bar = best path's economic profit. Bars with an absolute amount under one dollar are hidden.
- Toggle above the tiles: "All runs" (`metrics`) or "Surviving runs only" (`metricsSurvivorsOnly`; disabled if null).

Panel 4: Decision tree

- Layout: one column per decision point, ordered by (day, order), left to right, 220px apart. Column header: "Day 10 · Chase Firm A". Each column lists its edges (options) as rounded boxes, 36px tall, 8px apart.
- Each option box shows its label, its `survivalProb` and its `economicProfit`. A thin connector runs from each box to the next column.
- **Highlighting:** boxes and connectors with `onBestPath: true` are full opacity, 3px accent border (#2563EB), bold label. Every other box and connector is drawn at opacity 0.3 with a 1px grey border.
- Clicking an option box pins that option's `bestPathId`: its box gets a dashed grey border at full opacity, Panels 2 and 3 add that path's lines in the pinned colour, and a chip "Comparing: <plan sentence>" appears with a × to clear. Only one pinned path at a time. "Reset" clears it.
- Frontier paths (`frontierPathIds`) get a small star on their option boxes.

Panel 5: Controls and assumptions

- Sliders: shock amount (0 to `threshold.maxShock`, step 1,000), shock announce day and due day (1 to horizonDays, due ≥ announce), starting cash (0 to 200,000, step 1,000).
- Editable tables (number inputs, validated with the field reference ranges): customers (meanDaysToPay, offer acceptance alpha and beta), actions (multiplier, cost, late fee rate, churn probabilities), business (gross margin, elasticity, credit repay level), objective (survival target, churn months after end).
- Toggle "Deterministic" (runs the engine's deterministic mode). Buttons "Load scenario JSON" and "Download scenario JSON", so any business can be loaded and the edited one saved.
- Any change re-runs the engine in the worker after 300ms with no further change. While running, keep showing the previous result dimmed to 50% opacity with a progress bar from the worker's stage messages. An invalid input shows the field's error message and does not run.

States

- **Loading (first run):** skeleton blocks in each panel.
- **Engine error or invalid file:** one red banner with the error text; panels keep the last valid result.
- **Accessibility:** colour is always paired with text or shape; every control works with the keyboard; each chart has a text title and axis labels.

Tech stack and modules

Use TypeScript end to end with React and Vite, running the simulation in a web worker. No backend is needed, sliders update live, and the whole demo runs from one static site. 192 paths × 1,000 runs × 90 days is about 17 million day-steps, which a worker handles in a few seconds.

Module	Responsibility
`src/model/types.ts`	Scenario, Action, Path, Trace and Result types, plus validateScenario() written with zod to enforce the field reference ranges
`src/model/scenario.json`	The example business from this spec (one of any number of scenario files under src/model/scenarios/)
`src/sim/rng.ts`	Seeded generator (mulberry32) and Beta/Uniform helpers
`src/sim/engine.ts`	`simulate(scenario, path, rng)`: the daily loop from the accounting rules
`src/sim/actions.ts`	Action definitions: multipliers, costs, churn
`src/sim/optimise.ts`	Enumerate paths, run N runs each, summarise, rank
`src/sim/threshold.ts`	Bisection for largest shock and extra buffer; survival curve
`src/sim/worker.ts`	Web worker wrapper; posts progress and the result JSON
`src/ui/VerdictBar.tsx`	Panel 1
`src/ui/Timeline.tsx`	Panel 2 (Recharts)
`src/ui/DecisionTree.tsx`	Panel 3 (SVG, hand-laid out)
`src/ui/Controls.tsx`	Panel 4
`tests/*.test.ts`	Vitest unit and scenario tests

Keep `engine.ts` free of UI code and of `Math.random`. Every random draw goes through `rng.ts`, so tests are reproducible.

Pinned versions: React 18, Vite 5, TypeScript 5 in strict mode, Recharts 2, zod 3, Vitest 1. No other runtime dependencies without a written reason. Do not add a backend or Python for this prototype; the engine must run in the browser worker so sliders update without a server.

Tests and validation

All tests below must pass before any UI work starts. Expected values were worked out by hand from the example file; a mismatch means the engine is wrong, not the test.

Hand-calculated checks (deterministic mode, example file, shock amount set to 0, default path)

Check	Expected	How it is derived
Firm B payment day	day 7, S$780.00	0.9^7 = 0.478 ⩽ 0.5; discount window ended day −19; no late fee on default path
Firm C payment day	day 21, S$196.00	30 days: (29/30)^21 ⩽ 0.5 first at k = 21; inside window (day −15 + 45 = 30), 2% off
Firm A open invoice	not paid by day 90	(179/180)^k ⩽ 0.5 first at k = 125
Cash on day 23	S$20,476.00	20,000 + 780 + 196 − 500
Invoices raised	day 30, 60, 90: A S$600, B S$100 each	3 cycles
B's day-30 and day-60 invoices	paid day 37 and day 67, S$98.00 each	7 days after becoming eligible; inside 45-day window
Cash on day 90	S$10,672.00	20,000 + 780 + 196 − 500 + 98 − 10,000 + 98
Revenue	S$2,100.00	(600 + 100) × 3
Profit	S$122.00	gross profit 630 − discounts 8 − utilities 500

Zero-payment check

Set every customer's meanDaysToPay to 1,000,000 and the shock to 0. Cash is S$20,000.00 on days 1–22, S$19,500.00 on day 23, and S$9,500.00 from day 60.

Unit checks

- [ ] Late fee: invoice S$780, due day −4, fee rate 1% per month from day 10, paid day 20 gives a fee of S$2.60.
- [ ] Price cut: firm A invoice with priceMultiplier 0.8 and elasticity 0.6 is S$537.60.
- [ ] Churn cost: firm A churned, 9 months after end, margin 0.30 gives S$1,620.00.
- [ ] Discount offer: an accepted invoice is paid exactly payWithinDays after the decision day, at the larger of the offer and any still-valid early-pay discount.
- [ ] Validation: removing any required field, or setting a value outside its range, refuses to run with a message naming the field.
- [ ] Path count: the example produces exactly 192 paths (4 × 3 × 8 × 2); a scenario with more than 5,000 refuses to run.

Invariant checks (random mode, every path, every run)

- [ ] Accounting identity holds every day to within S$0.01.
- [ ] Profit bridge items add up to the difference in mean economic profit to within S$0.01.
- [ ] No invoice is collected twice; no payable is paid twice; receivables and payables never go negative.
- [ ] Credit drawn never exceeds the sum of limits.
- [ ] Same seed gives byte-identical `Result` JSON (excluding `elapsedMs`).
- [ ] Two paths with identical choices up to day d have identical snapshots up to day d − 1 in every run (common random numbers).

Behaviour checks (random mode, example file)

- [ ] S$30,000 shock: every path has survivalProb 0; `extraBufferForChosenShock` is a positive number.
- [ ] Best-path survival never rises as the shock amount rises (curve is non-increasing within 0.02 for noise).
- [ ] Best-path survival never falls as cash0 rises (same tolerance).
- [ ] A letter of demand on firm A gives a lower mean payment day for that invoice than doing nothing, and a higher mean churn cost.

After any change to scoring, also inspect the top 5 paths by hand. The earlier prototype's optimiser found two scoring loopholes that inflated results by about 25%; treat any surprisingly good path as a possible bug until it is explained.

Build order and out of scope

Build in this order; each step ends with its tests passing.

1. Types, `scenario.json`, seeded RNG.
2. Daily engine with no actions; deterministic hand-calculation and identity tests.
3. Probability models (hazard, discount acceptance, churn, price) and the action menu.
4. Enumeration optimiser and ranking; reproducibility and common-random-number tests.
5. Threshold search and survival curve; shock and monotonicity tests.
6. Web worker and the result JSON contract.
7. UI: verdict bar, then timeline with scrubber, then decision tree with highlighting, then controls.
8. Polish: plain-language plan sentence, deterministic toggle, phone layout.

Out of scope for this prototype:

- The LLM agent (setting objectives from the owner's words, explaining results).
- Learning parameters from real transaction history (priors only for now).
- Importing data from accounting software.
- Time value of money, taxes, multiple currencies.
- Optimisers other than enumeration (CEM, RL).